Study guide: Advanced Git

Command	Explanation & Link
git commit -a	$ git commit -a automatically stages the files that have been locally modified. New files which have not been published yet are not affected.
git log -p	$ git log -p produces patch text that displays the lines of code that were changed in each commit in the current repo.
git show	$ git show shows you one or more object(s) such as blobs, trees, tags, and commits.
git diff	$ git diff is similar to the Linux `diff` command, and can show the changes between commits, changes between the working tree and index, changes between two trees, changes from a merge, and so on.
git diff --staged	$ git diff --staged is an alias of $ git diff --cached, which shows all staged files compared to the named commit.
git add -p	$ git add -p allows a user to interactively review patches before adding to the current commit.
git mv	$ git mv is similar to the Linux `mv` command. This command can move or rename a file, directory, or symlink.
git rm	$ git rm is similar to the Linux `rm` command. This command deletes or removes a file from the working tree.

There are many useful git command summaries online as well. Please take some time to research and study a few, such as this one.

.gitignore files

.gitignore files are used to tell the git tool to intentionally ignore some files in a given Git repository. For example, this can be useful for configuration files or metadata files that a user may not want to check into the master branch.

When writing a `.gitignore` file, there are some specific formats which help tell Git how to read the text in the file. For example, a line starting with # is a comment; a slash / is a directory separator. Visit https://git-scm.com/docs/gitignore to see more examples.

This GitHub repository offers some examples of configurations which are often included in a `.gitignore` file. These examples include: compiled sources, packages, logs, databases, and OS generated files.